App Development Project Report

App Dev Project Report

1. Student Details

Name: Shubham Kumar
Roll Number: 23F2002767
Email: 23f2002767@ds.study.iitm.ac.in
About Me: I am a dual degree student pursuing B.Tech at Silicon University and a BS in Data Science at IIT Madras. I am interested in backend development, system design, and scalable web application architecture.

---

2. Project Details

Project Title: INSTITUTE PLACEMENT PORTAL

Problem Statement:

Institutes require a centralized system to manage campus placement activities involving students, companies, and placement drives. Manual processes such as spreadsheets and emails make it difficult to track applications, approvals, and placement outcomes. This project implements a web-based Placement Portal that allows Admin, Companies, and Students to interact through a structured workflow for company approvals, placement drive creation, student applications, and recruitment decisions.

Approach:

The app was built using Flask as the backend framework with separate modules for Admin, Company, and Student workflows. It utilizes SQLAlchemy for database management and features secure, role-based user authentication, dynamic job listing/filtering, and real-time application tracking.

The system allows:

- Admin to approve companies and placement drives
- Companies to create drives and manage applications
- Students to view and apply for approved placement drives

Business logic is organized into service layers while routes handle request-response processing. SQLAlchemy ORM is used for database interaction with SQLite.

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist with code suggestions, documentation structuring, and understanding framework patterns. Approximately **10–15%** assistance was used for guidance and formatting. All system design decisions, implementation logic, debugging, and integration were performed manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database for storing user data
Flask-Login	User authentication and session management

---

5. Database Schema / ER Diagram

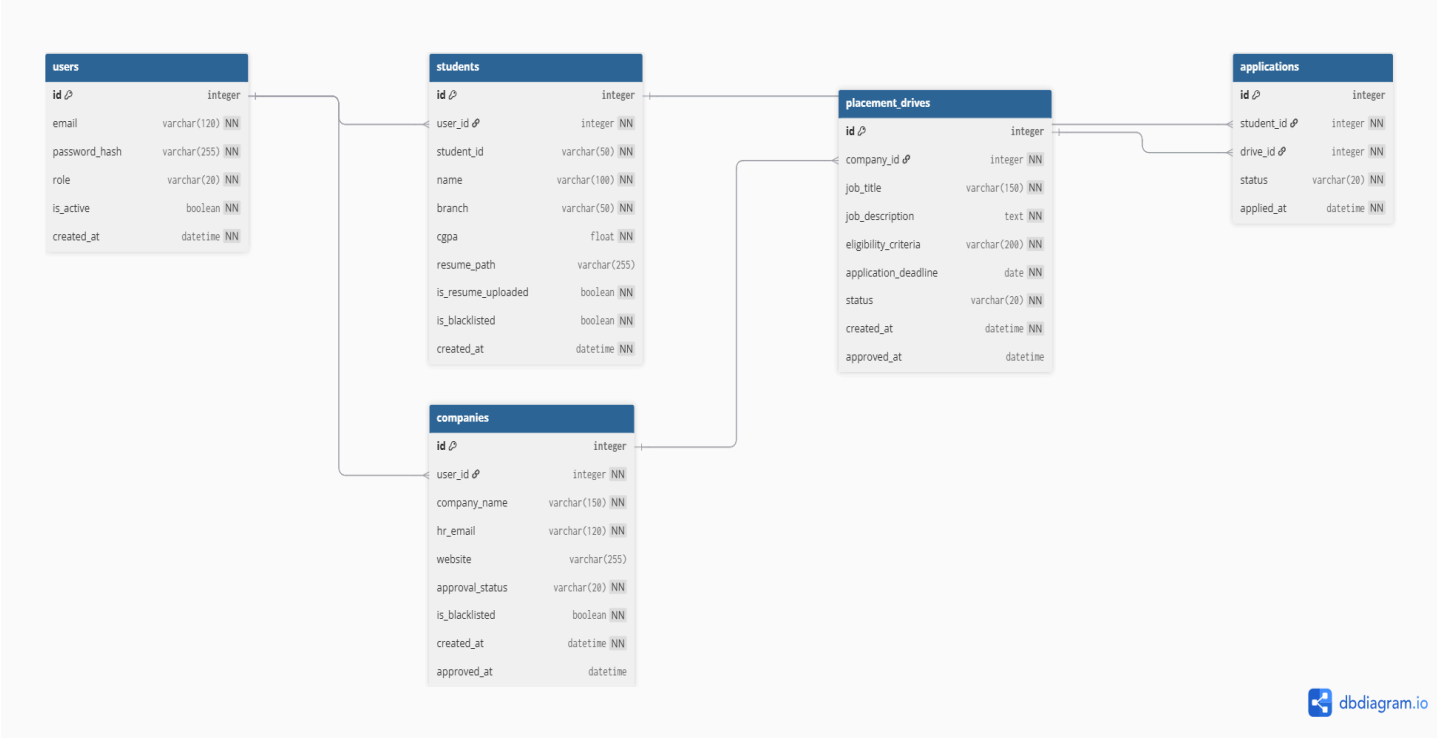
Tables:

1. **users - id, email, password_hash, role, is_active**
2. **students - id, user_id, student_id, name, branch, cgpa, resume_path, is_resume_uploaded**

3. **companies - id, user_id, company_name, hr_email, website, approval_status, is_blacklisted**
4. **placement_drives - id, company_id, job_title, job_description, eligibility_criteria, application_deadline, status**
5. **applications - id, student_id, drive_id, applied_at, status**

Relationships:

- One-to-One → User → Student
- One-to-One → User → Company
- One-to-Many → Company → PlacementDrive
- One-to-Many → Student → Application
- One-to-Many → PlacementDrive → Application



---

6. API Resource Endpoints

Endpoint	Method	Description
`/api/auth/login`	POST	Authenticate user and generate session
`/api/student/apply/<drive_id>`	POST	Apply to Placement Drive
`/api/auth/register-student`	GET / POST	Student Registration
`/api/admin/approve-drive/<drive_id>`	POST	Approve Placement Drive

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features (optional)

Architecture Overview:

- **run.py** – Main Flask application entry point
- **/models** – database models using SQLAlchemy
- **/blueprints** – Role based Route Modules
- **/templates** – Jinja2 based UI for Admin,Company and Student
- **/utils** – Constants and Validation helpers

The project follows a modular Flask architecture using the application factory pattern. The system is organized into separate layers including models for database entities, services for business logic, blueprints for routing, and templates for the user interface. This separation ensures that routes remain lightweight while core business logic is handled by the service layer. The architecture separates business logic and presentation layers, making the system easily extensible for REST APIs and modern frontend frameworks such as React.

Implemented Features:

- User authentication (student, company, admin)
- Admin approval of companies and placement drives
- Company dashboard for drive management
- Student dashboard to view and apply to drives

- Duplicate application prevention
- Resume upload system
- Application status tracking
- Placement history tracking

Additional Features (to be implemented in Future):

- Export user logs as CSV
- Notifications
- Placement Analytics Dashboard
- ERP Integration
- Forget Password
- Interview Scheduling System

---

8. Video Presentation

Drive Link:

https://drive.google.com/drive/folders/1UKSyBrDa0noOCdYON7ioKd-oj12liDZh?usp=drive_link

(Accessible to all with "View" permission.)

GitHub Link:

https://github.com/Coder-001-csk/MAD-1-Official-Project-Placement-Portal

---